Lab 5: Static Code Analysis

Name	Arundhati K
SRN	PES1UG24CS083
Section	B
Course	SE Lab 2
Program	tower_maintenance.py
Repository	github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083 (folder lab-5)

1. Introduction

Static code analysis examines source code without executing it to detect bugs, security vulnerabilities and style violations. Issues are caught early in development, when they are cheapest to fix, and coding standards are enforced consistently across a codebase.

Tool	Focus	Typical checks
Pylint	Code quality and logical errors	Naming conventions, missing docstrings, broad exception handling, global variables, unnecessary code. Produces a score out of 10.
Bandit	Security	SQL injection, unsafe deserialisation, hardcoded passwords, dangerous function calls. Rates each finding by severity and confidence.
Flake8	PEP 8 style	Line length, whitespace, blank lines, unused imports and syntax errors. Combines pycodestyle, pyflakes and mccabe.

The program analysed, `tower_maintenance.py`, is a maintenance tracker for telecom towers that stores records in an in-memory SQLite database.

Tool	Before	After
Pylint	5.14/10 (18 messages)	10.00/10
Bandit	4 issues (2 Medium, 2 Low)	No issues
Flake8	7 issues	0 issues

2. Issues Identified Before Fixing

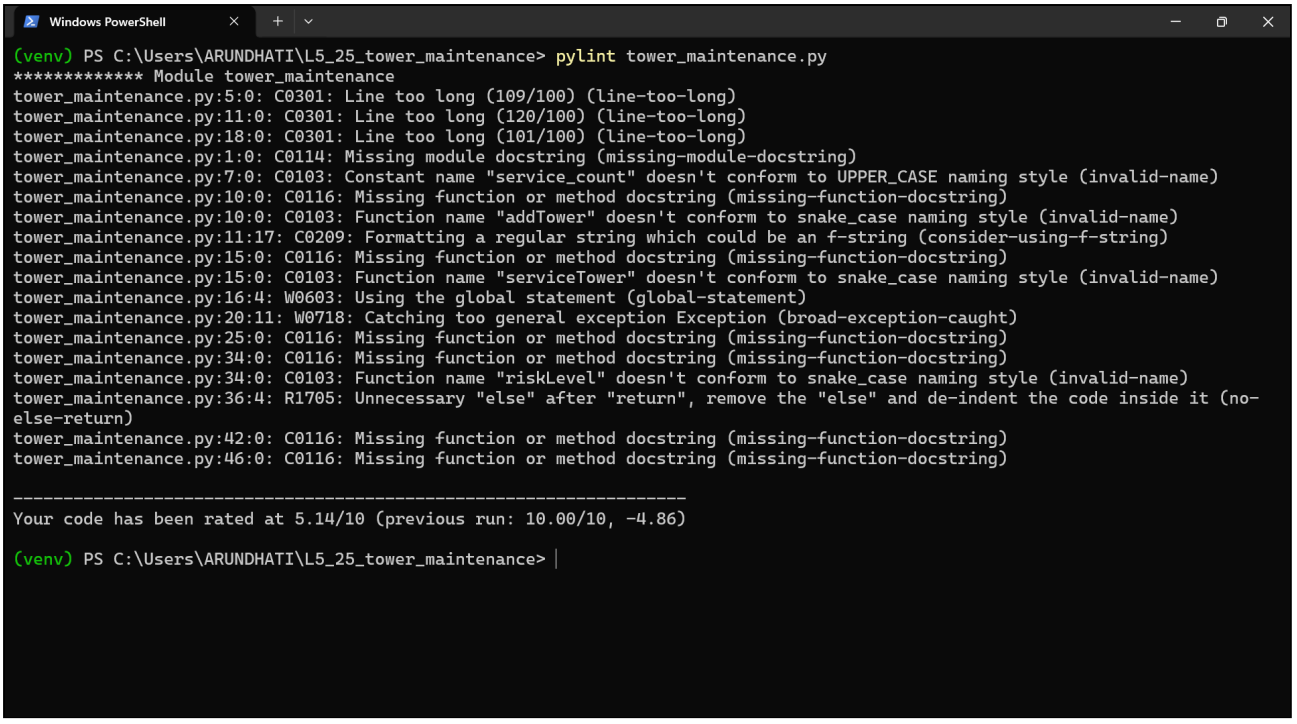


Figure 1: Pylint on the original code (score 5.14/10)

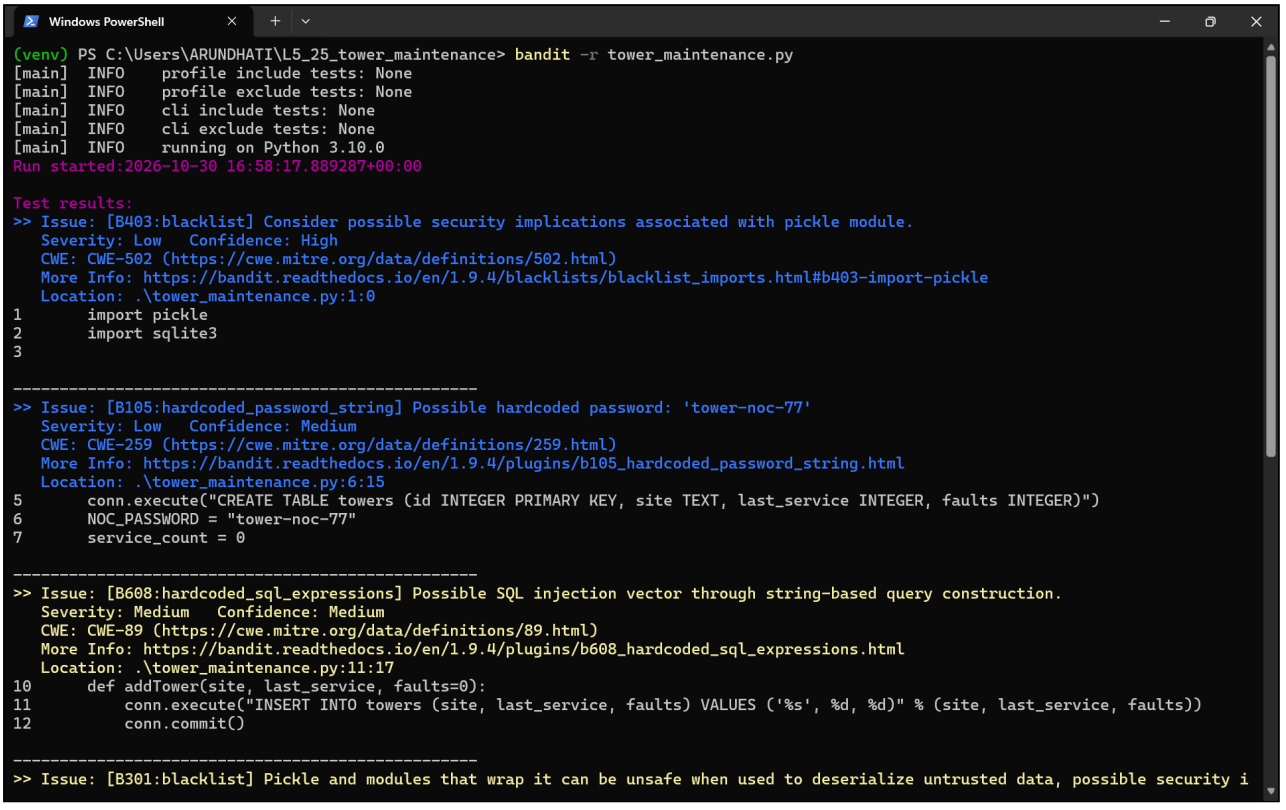


Figure 2: Bandit on the original code (part 1)

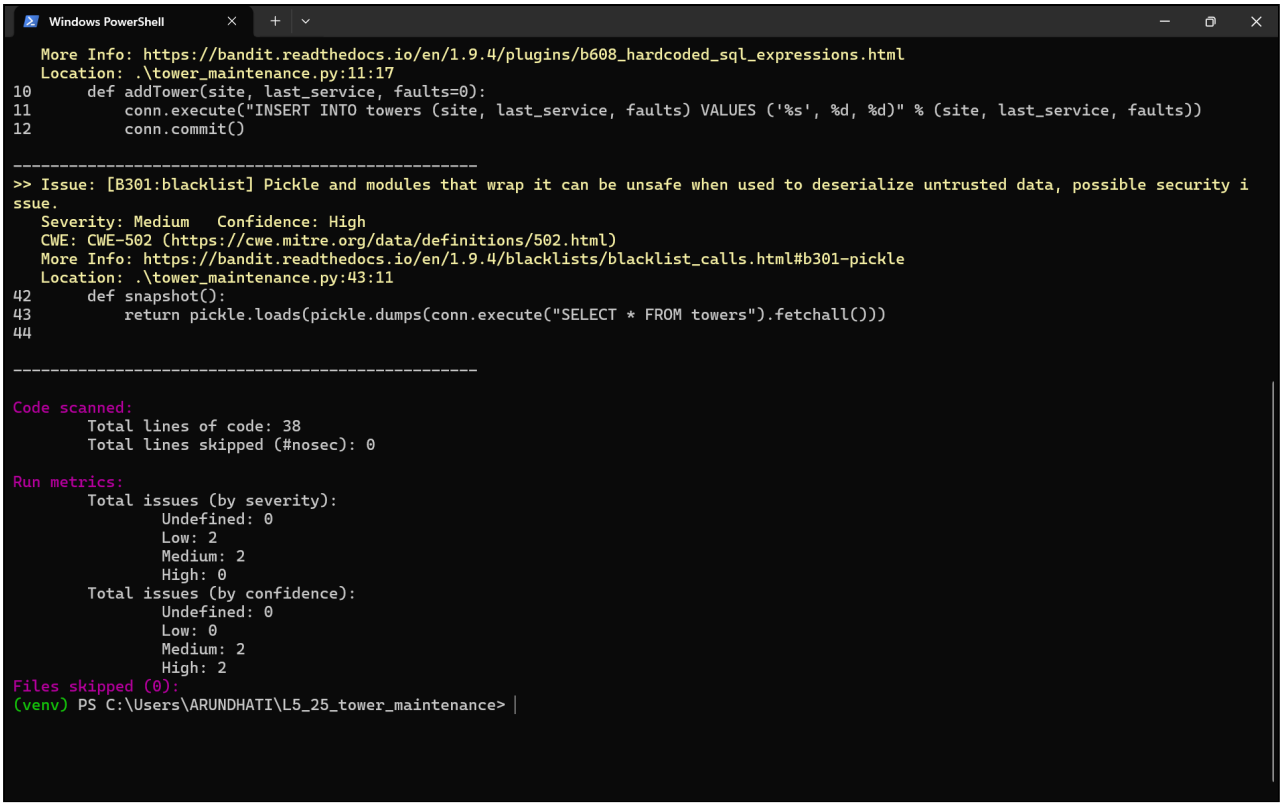


Figure 3: Bandit on the original code (part 2): 2 Medium, 2 Low

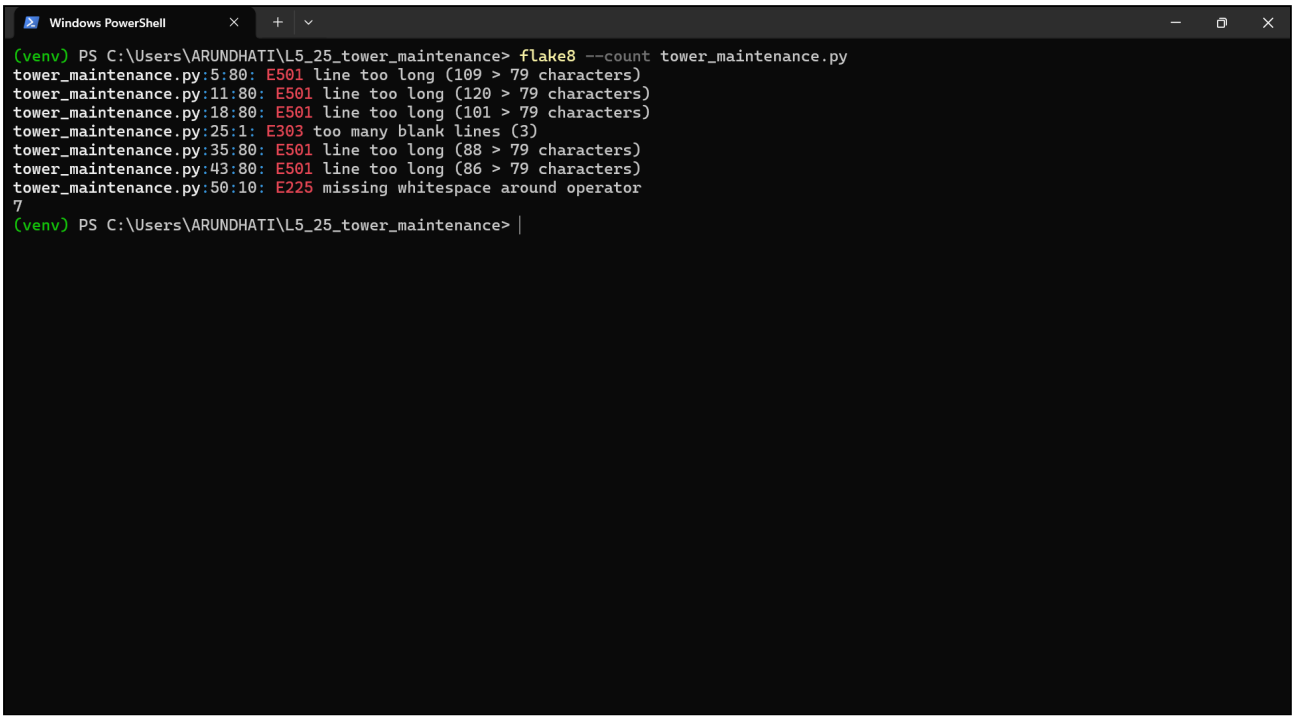


Figure 4: Flake8 on the original code (7 issues)

3. Issue Identification Table

Line numbers refer to the original file. High and medium severity issues are listed first.

#	Issue	Tool / Type	Line(s)	Description	Fix
1	SQL injection	Bandit B608 (Medium)	11	Query built with % string formatting. A site name containing a quote alters the SQL; the name O'Brien crashes the program.	Parameterised query with ? placeholders.
2	Unsafe deserialisation (pickle)	Bandit B301 (Medium)	43	snapshot() round-trips rows through pickle.dumps and pickle.loads, which can execute arbitrary code on untrusted data and serves no purpose here.	Return list(...) of the fetched rows.
3	Pickle import	Bandit B403 (Low)	1	Import of a module flagged as insecure.	Import removed.
4	Hardcoded password	Bandit B105 (Low)	6	NOC_PASSWORD stored in source code and never used.	Variable removed; secrets belong in environment variables.
5	Global statement	Pylint W0603	7, 16	Counter and database connection held as module-level globals.	State moved into a TowerTracker class.
6	Broad exception	Pylint W0718	20	except Exception hides programming errors behind "service failed".	Catch only sqlite3.Error.
7	Crash on unknown tower	Bug	35-36	riskLevel indexes row[0] when fetchone() returns None, raising TypeError.	Check for None and return "unknown".
8	Unnecessary else after return	Pylint R1705	36-39	Redundant else branch adds nesting.	else removed.
9	Non snake_case names	Pylint C0103	10, 15, 34	addTower, serviceTower and riskLevel violate PEP 8 naming.	Renamed to add_tower, service_tower, risk_level.
10	Constant naming	Pylint C0103	7	Mutable counter service_count at module level is treated as a constant.	Now the instance attribute self.service_count.
11	Missing docstrings	Pylint C0114, C0116	1, 10, 15, 25, 34, 42, 46	No module or function docstrings.	Docstrings added throughout.
12	Old-style formatting	Pylint C0209	11	% formatting used to build the query.	Removed by the parameterised query (an f-string would still be injectable).
13	Code runs on import	Code quality	54	main() is called unconditionally at module level.	Wrapped in if __name__ == "__main__":.
14	Line too long	Flake8 E501, Pylint C0301	5, 11, 18, 35, 43	Lines up to 120 characters; PEP 8 limit is 79.	Long statements wrapped inside brackets.
15	Too many blank lines	Flake8 E303	25	Three blank lines before overdue().	Reduced to two.
16	Missing whitespace around operator	Flake8 E225	50	total=len(...)	Changed to total = len(...).

4. Tool Reports Before and After

4.1 pylint_report_before.txt

```
************* Module tower_maintenance
tower_maintenance.py:5:0: C0301: Line too long (109/100) (line-too-long)
tower_maintenance.py:11:0: C0301: Line too long (120/100) (line-too-long)
tower_maintenance.py:18:0: C0301: Line too long (101/100) (line-too-long)
tower_maintenance.py:1:0: C0114: Missing module docstring (missing-module-docstring)
tower_maintenance.py:7:0: C0103: Constant name "service_count" doesn't conform to UPPER_CASE naming style
(invalid-name)
tower_maintenance.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
tower_maintenance.py:10:0: C0103: Function name "addTower" doesn't conform to snake_case naming style (in
valid-name)
tower_maintenance.py:11:17: C0209: Formatting a regular string which could be an f-string (consider-using
-f-string)
tower_maintenance.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
tower_maintenance.py:15:0: C0103: Function name "serviceTower" doesn't conform to snake_case naming style
(invalid-name)
tower_maintenance.py:16:4: W0603: Using the global statement (global-statement)
tower_maintenance.py:20:11: W0718: Catching too general exception Exception (broad-exception-caught)
tower_maintenance.py:25:0: C0116: Missing function or method docstring (missing-function-docstring)
tower_maintenance.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
tower_maintenance.py:34:0: C0103: Function name "riskLevel" doesn't conform to snake_case naming style (i
nvalid-name)
tower_maintenance.py:36:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the
code inside it (no-else-return)
tower_maintenance.py:42:0: C0116: Missing function or method docstring (missing-function-docstring)
tower_maintenance.py:46:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 5.14/10
```

4.2 pylint_report_after.txt

```
-----------------------------------
Your code has been rated at 10.00/10
```

4.3 flake8_report_before.txt

```
tower_maintenance.py:5:80: E501 line too long (109 > 79 characters)
tower_maintenance.py:11:80: E501 line too long (120 > 79 characters)
tower_maintenance.py:18:80: E501 line too long (101 > 79 characters)
tower_maintenance.py:25:1: E303 too many blank lines (3)
tower_maintenance.py:35:80: E501 line too long (88 > 79 characters)
tower_maintenance.py:43:80: E501 line too long (86 > 79 characters)
tower_maintenance.py:50:10: E225 missing whitespace around operator
```

4.4 flake8_report_after.txt

(File is empty. Flake8 produces no output when no issues are found.)

4.5 bandit_report_before.txt

```
Run started:2026-10-28 17:43:03.836632+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: ./tower_maintenance.py:1:0
1       import pickle
2       import sqlite3
3

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'tower-noc-77'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: ./tower_maintenance.py:6:15
5       conn.execute("CREATE TABLE towers (id INTEGER PRIMARY KEY, site TEXT, last_service INTEGER, fault
s INTEGER)")
6       NOC_PASSWORD = "tower-noc-77"
7       service_count = 0

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query const
ruction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: ./tower_maintenance.py:11:17
10      def addTower(site, last_service, faults=0):
11          conn.execute("INSERT INTO towers (site, last_service, faults) VALUES ('%s', %d, %d)" % (site,
last_service, faults))
12          conn.commit()

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrust
ed data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: ./tower_maintenance.py:43:11
42      def snapshot():
43          return pickle.loads(pickle.dumps(conn.execute("SELECT * FROM towers").fetchall()))
44

--------------------------------------------------

Code scanned:
        Total lines of code: 38
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 2
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 2
                High: 2
Files skipped (0):
```

4.6 bandit_report_after.txt

```
Run started:2026-10-28 17:44:03.170812+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 68
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

5. Results After Fixing

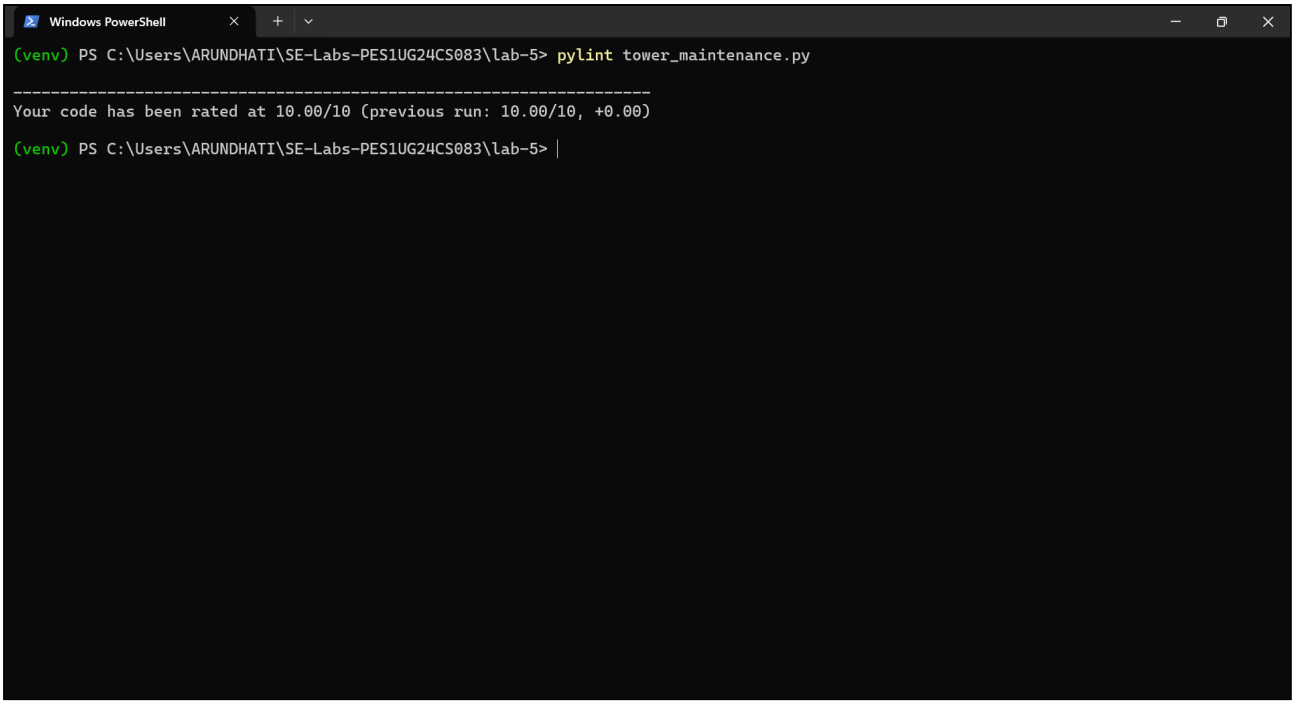


Figure 5: Pylint on the fixed code (score 10.00/10)

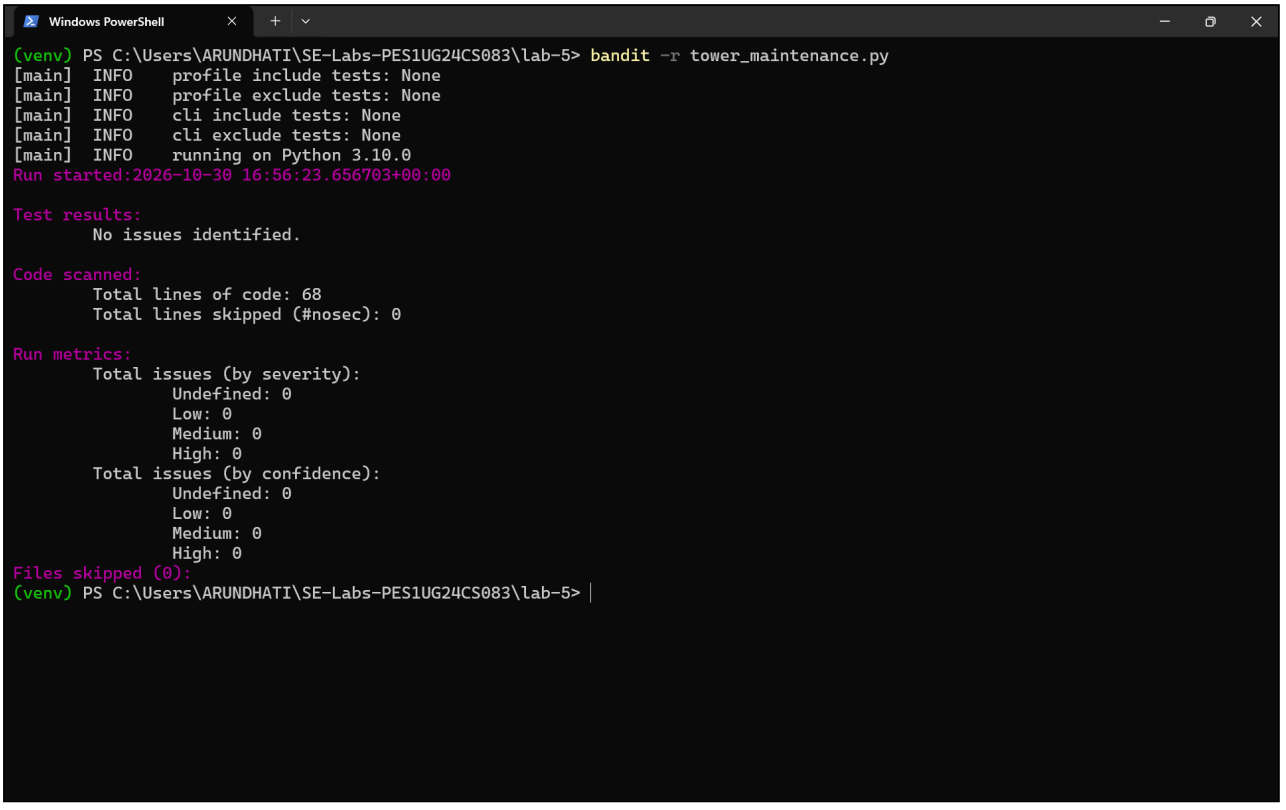


Figure 6: Bandit on the fixed code (no issues identified)

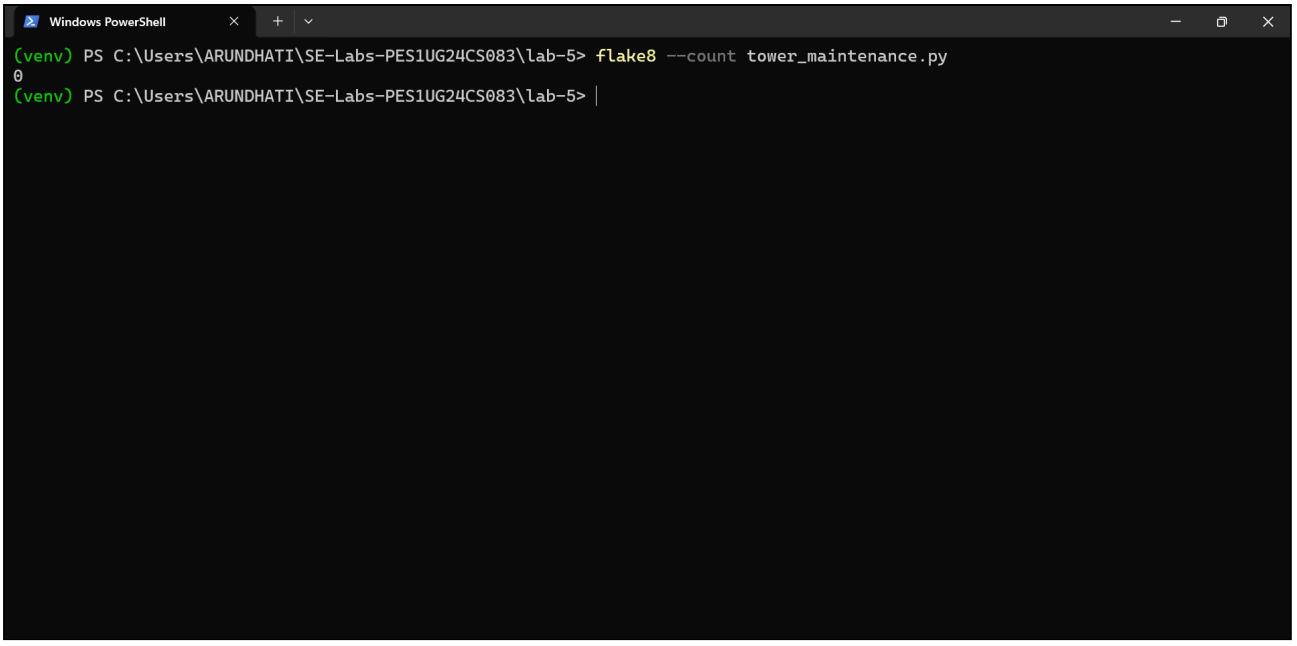


Figure 7: Flake8 on the fixed code (0 issues)

6. Reflection

Q1. Which issues were the easiest to fix, and which were the hardest? Why?

The Flake8 issues were the easiest. Long lines, an extra blank line and missing whitespace are purely formatting changes and cannot affect behaviour. Renaming functions and adding docstrings were similarly mechanical.

The hardest was removing the global statement (W0603). The connection and counter were shared by every function, so the fix required restructuring the code into a TowerTracker class and updating every call, then confirming the output was unchanged. The SQL injection fix was short but required care: Pylint's suggestion to use an f-string would have kept the vulnerability, and the correct fix was a parameterised query.

Q2. Did the static analysis tools report any false positives? If so, describe one example.

Pylint's C0103 warning on service_count was misleading. It assumes a module-level variable is a constant and asks for UPPER_CASE, but service_count changes on every service. Renaming it would have silenced the warning while making the code less clear. The underlying problem was global state, which was fixed by moving the counter into the class. All Bandit findings were genuine.

Q3. How would you integrate static analysis tools into your actual software development workflow?

The tools should run automatically rather than on demand:

- **Editor:** Pylint and Flake8 integrated into the IDE to flag issues while writing code.
- **Pre-commit hooks:** Flake8 and Bandit run before each commit, so issues never reach the repository.
- **Continuous integration:** all three tools run on every push and pull request. The build fails on any Bandit finding of medium or high severity, or a Pylint score below an agreed threshold such as 9.5.

Q4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

- **Security:** the SQL injection vulnerability and unsafe pickle use were removed. Bandit reports no issues.
- **Robustness:** two crashes were fixed: inserting a site name containing an apostrophe, and requesting the risk level of a non-existent tower. Unexpected errors are no longer silently suppressed.
- **Readability:** PEP 8 naming, docstrings and encapsulated state. The Pylint score rose from 5.14/10 to 10.00/10 and Flake8 reports zero issues.
- **Reusability:** the main guard allows the module to be imported without running the demonstration.

The program output is identical before and after: ['Ridge-A'] high 2 1.

7. Git Commits

Repository: github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083, folder lab-5.

Commit	Changes
`d921615`	Original script and before reports (baseline)
`52fd554`	Bandit fixes: issues 1-4
`1d6f0b7`	Pylint fixes: issues 5-13
`4479f3a`	Flake8 fixes: issues 14-16, after reports, issue table and reflection